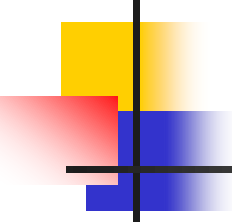


Outline

- Why R, and R Paradigm
- References, Tutorials and links
- R Overview
- R Interface, Rstudio, R Script
- Commands, objects and functions
- Points to note
- R Working directory and Workspace
- Help
- R-architecture and R Packages
- Data types



Why R?

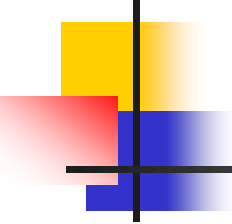
It's free!

It runs on a variety of platforms including Windows, Unix and MacOS.

It provides an unparalleled platform for programming new statistical methods in an easy and straightforward manner.

It contains advanced statistical routines not yet available in other packages.

It has state-of-the-art graphics capabilities.

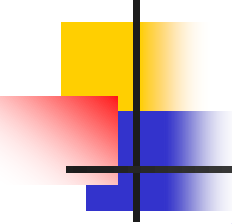


R paradigm is different

Rather than setting up a complete analysis at once, the process is highly interactive.

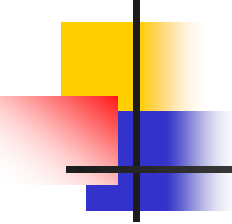
You run a command (say fit a model), take the results and **process it through another command** (say a set of diagnostic plots), **take those results and process it through another command** (say cross-validation), etc.

The cycle may include **transforming the data**, and looping back through the whole process again. You stop when you feel that you have fully analyzed the data.



How to download?

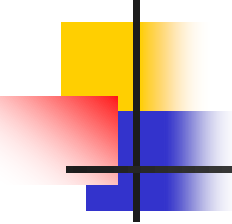
- Google it using R or CRAN
(Comprehensive R Archive Network)
 - <http://www.r-project.org>
- Instructions via this link: <https://rstudio-education.github.io/hopr/starting.html>
- **Rstudio:** helps you write in R makes using R much easier.
 - Freely available at <http://www.rstudio.com/ide>
 - Just click the “Download RStudio” button and follow the simple instructions that follow.



Tutorials

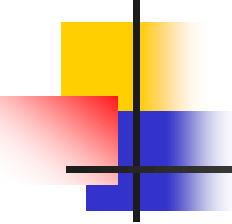
Each of the following tutorials are in PDF format.

- P. Kuhnert & B. Venables, [An Introduction to R: Software for Statistical Modeling & Computing](#)
- J.H. Maindonald, [Using R for Data Analysis and Graphics](#)
- W.J. Owen, [The R Guide](#)
- D. Rossiter, [Introduction to the R Project for Statistical Computing for Use at the ITC](#)
- W.N. Venables & D. M. Smith, [An Introduction to R](#)



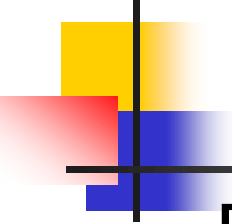
Web links

- [Dave Robert's Excellent Labs](#) on Ecological Analysis
- [Excellent Tutorials by David Rossitier](#)
- [Excellent tutorial an nearly every aspect of R](#) (c/o Rob Kabacoff)
- [Introduction to R by Vincent Zoonekynd](#)
- [Cookbook for R \(cookbook-r.com\)](#)
- [Data Manipulation Reference](#)



Web links

- [R time series tutorial](#)
- [R Concepts and Data Types](#)
presentation by Deepayan Sarkar
- [Interpreting Output From lm\(\)](#)
- [An Introduction to R](#)
- [Import / Export Manual](#)
- [R Reference Cards](#)
- [KickStart](#)
- [Hints on plotting data in R](#)



R Overview

R is a comprehensive statistical and graphical programming language and is a dialect of the S language:

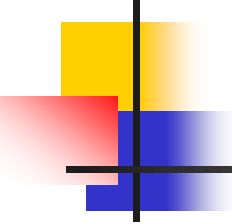
1988 - S2: RA Becker, JM Chambers, A Wilks

1992 - S3: JM Chambers, TJ Hastie

1998 - S4: JM Chambers

R: initially written by Ross Ihaka and Robert Gentleman at Dep. of Statistics of U of Auckland, New Zealand during 1990s.

Since 1997: international “R-core” team of 15 people with access to common CVS archive.



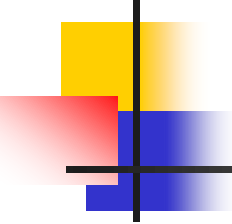
R Overview

You can enter commands one at a time at the command prompt (`>`) or run a set of commands from a source file.

There is a wide variety of data types, including vectors (numerical, character, logical), matrices, dataframes, and lists.

To quit R, use

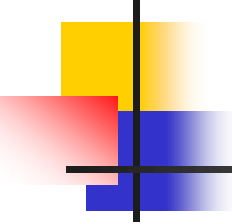
`q()`



R Overview

Most functionality is provided through built-in and user-created functions and all data objects are kept in memory during an interactive session.

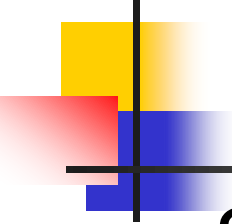
Basic functions are available by default. Other functions are contained in packages that can be attached to a current session as needed



R Overview

A key skill to using **R** effectively is learning how to use the built-in help system. Other sections describe the working environment, inputting programs and outputting results, installing new functionality through packages and etc.

A fundamental design feature of **R** is that the output from most functions can be used as input to other functions. This is described in reusing results.



R Interface

Start the R system, the main window (RGui) with a sub window (R Console) will appear

In the `Console' window the cursor is waiting for you to type in some R commands

Your First R Session: R GUI

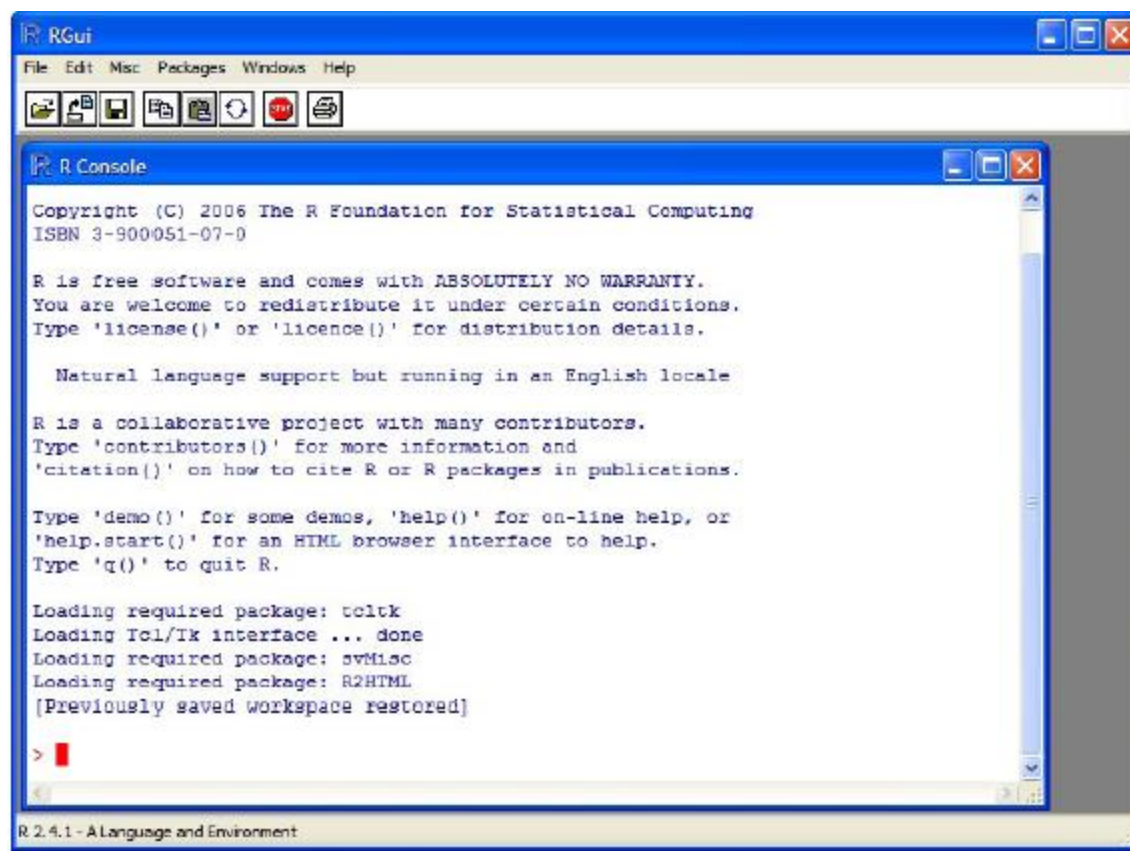


Figure 1.1: The R system on Windows

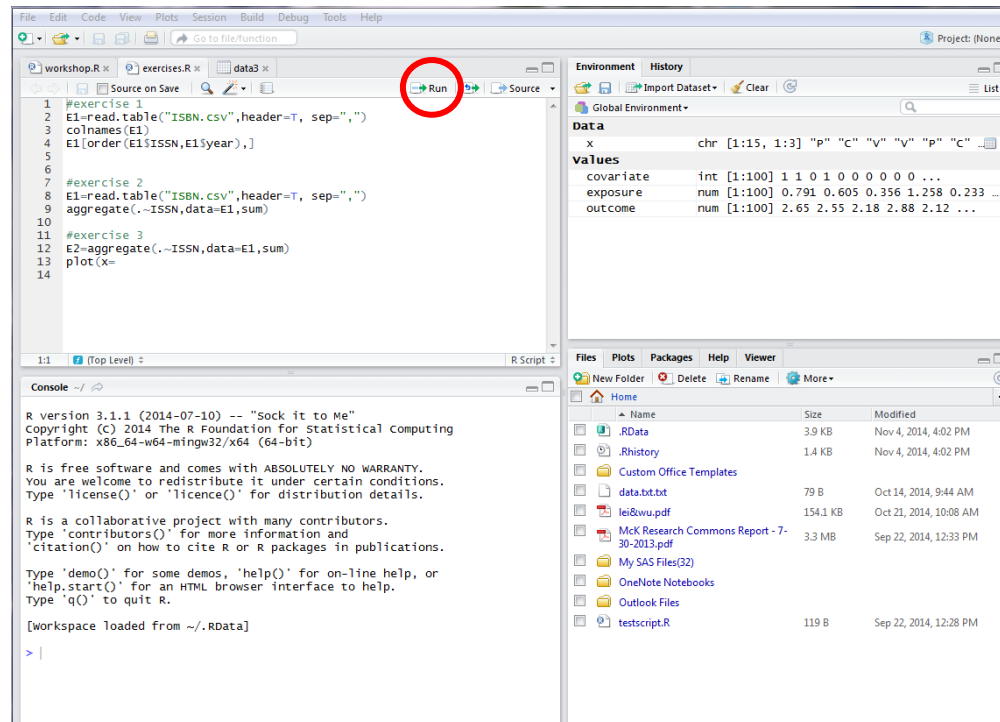
Basic Editor Elements (of Rstudio)



Source editor: Text editor for scripts.



Console: Execute code and examine the log.
-- Can also type commands here too!



Objects: names, properties, and variables

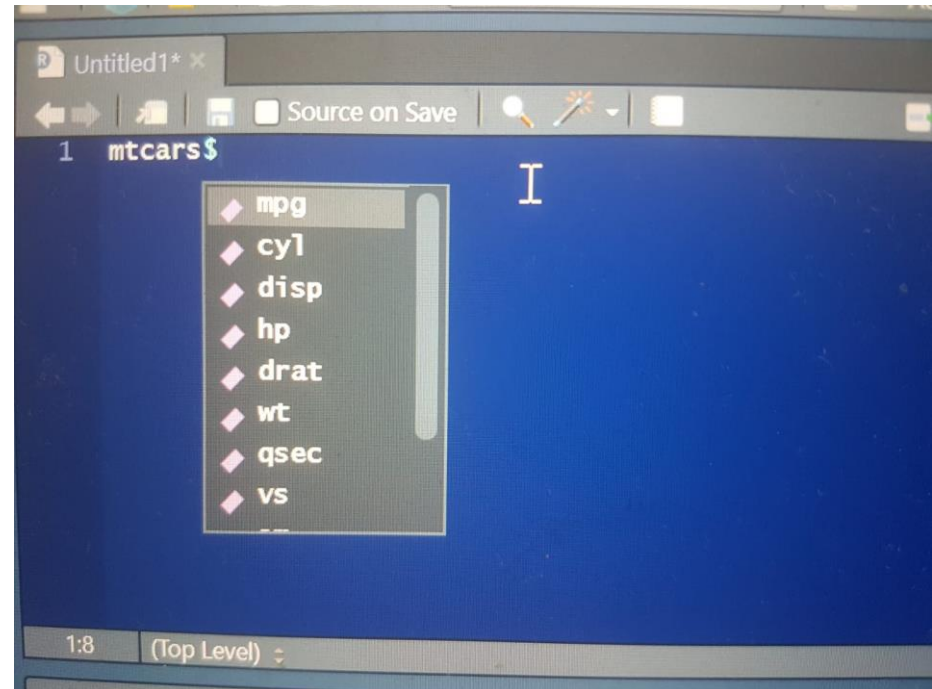
Library: File manager, plots, help files

RStudio: why?

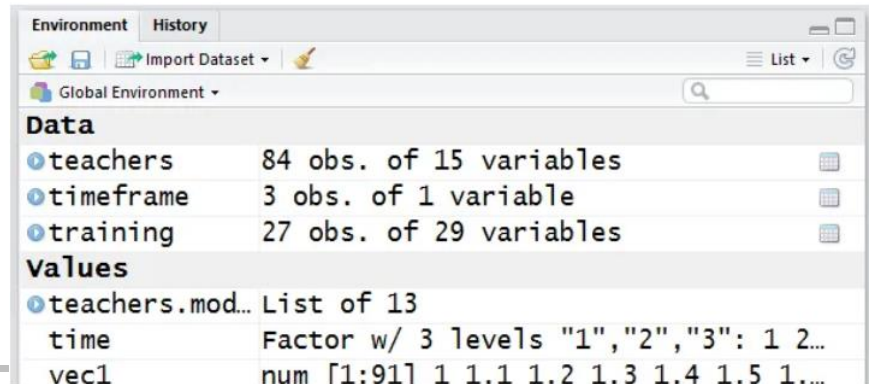
1) RStudio makes it easy to write scripts.

- you can see both your script and the results in your console when you run your syntax.
- ability to call up potential syntax options while you are writing just

- E.g. am trying to access a variable in a data set called "mtcars", but I haven't memorized the variable names
- I type mtcars then symbol \$



RStudio: why?



2) RStudio: convenient to view and interact with the objects stored in your environment.

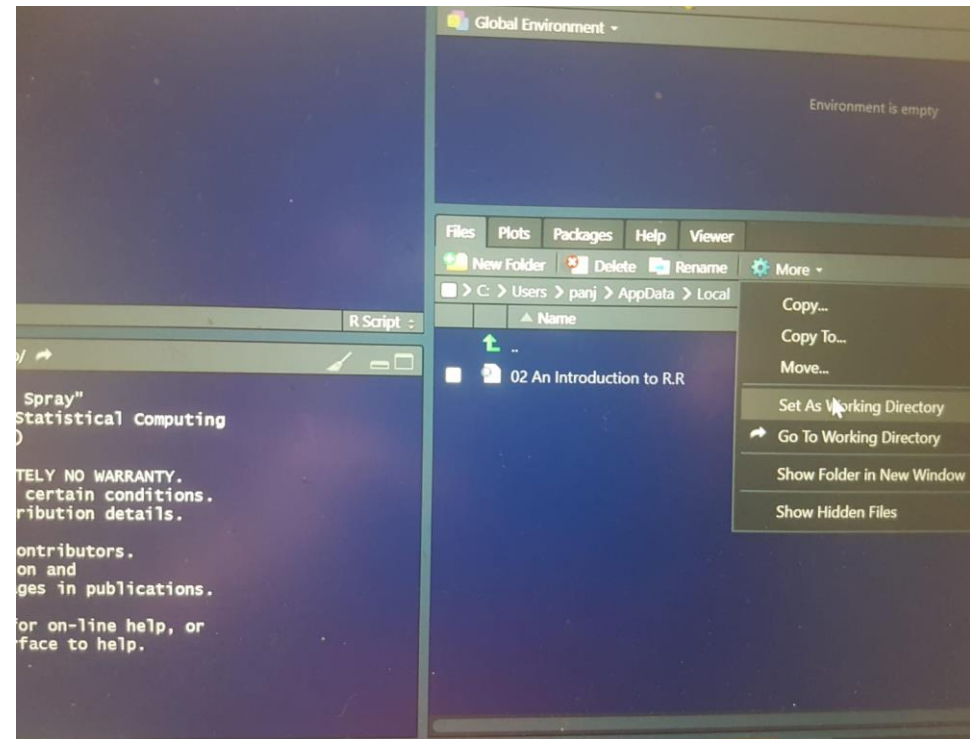
- In the basic R GUI, you can always list the objects you have stored in your environment. But RStudio has a very useful “Environment” window available.
- This shows all of the objects that you have stored, including data; scalars, vectors, and matrices; model outputs; etc., along with a summary of the information that is stored in those objects.
- You can even click on your data sets directly to open them and view them as spreadsheets.

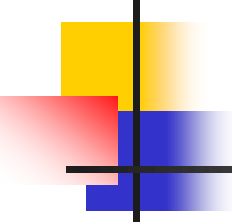
RStudio: why?

3) RStudio makes it easy to set your working directory and access files on your computer

4) RStudio makes graphics much more accessible for a casual user.

- Easy to save graphics as you go.
- You can easily click back and forth between plots



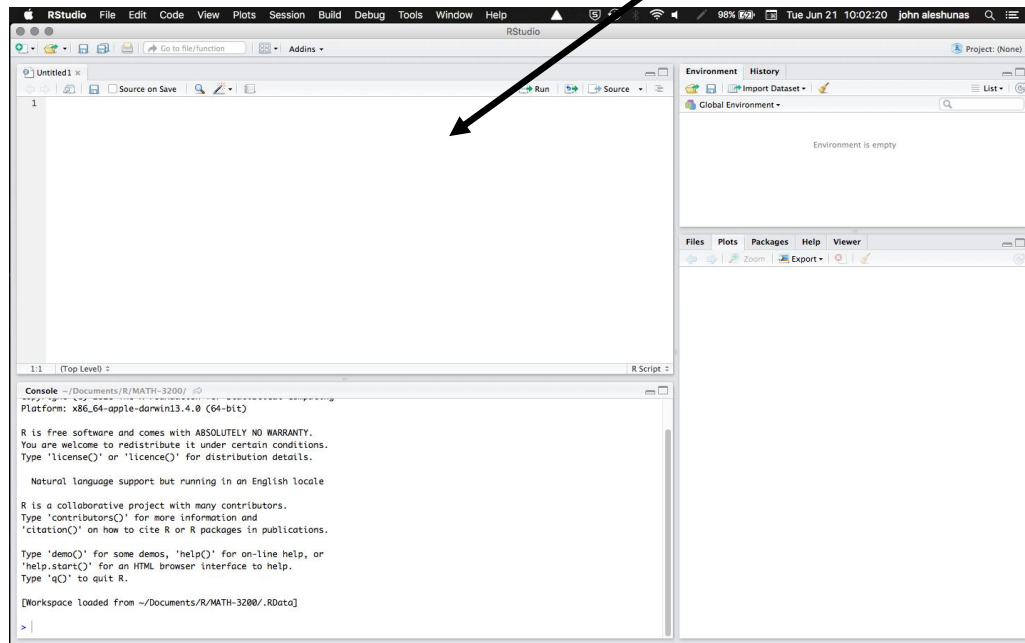


RStudio

Capabilities include:

- Administrators can install several versions of R and specify a global default version as well as per-user or per-group default versions.
- Users can switch between any of the available versions of R as they like.
- Users can specify that individual R projects remember their last version of R and always use that version until explicitly migrated to a new version.

Get started: R script

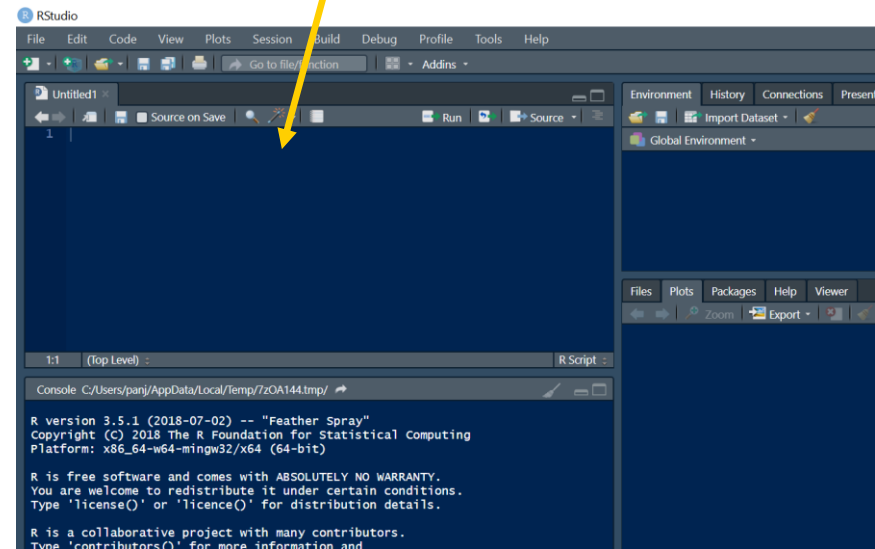
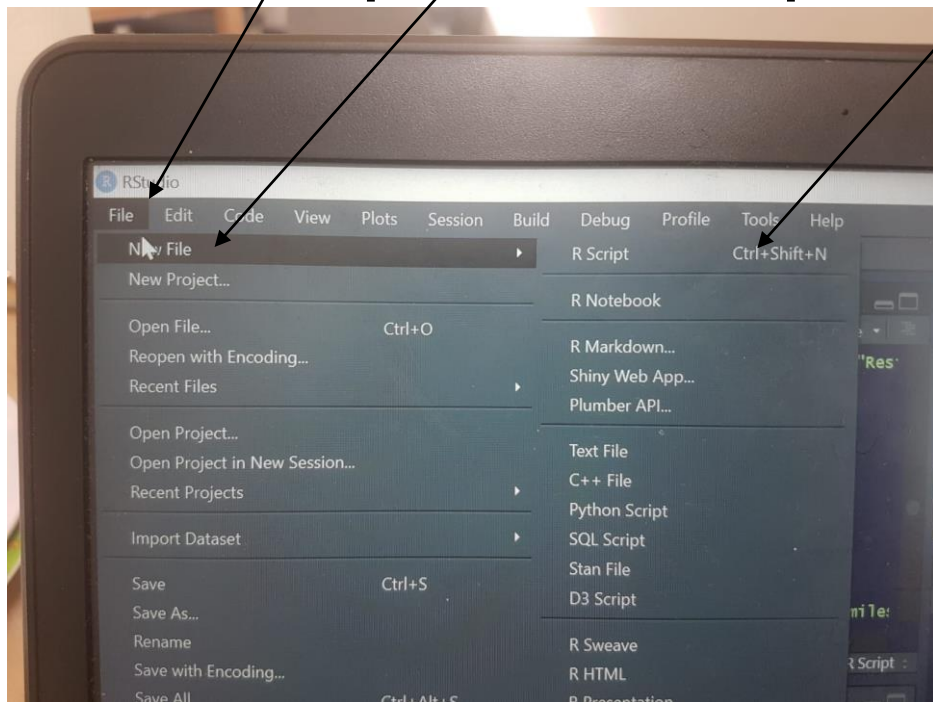


- **Why Using R Script rather than inputting commands in the console directly?**

1. To save your codes for later reference, usage and sharing
2. Easy to edit

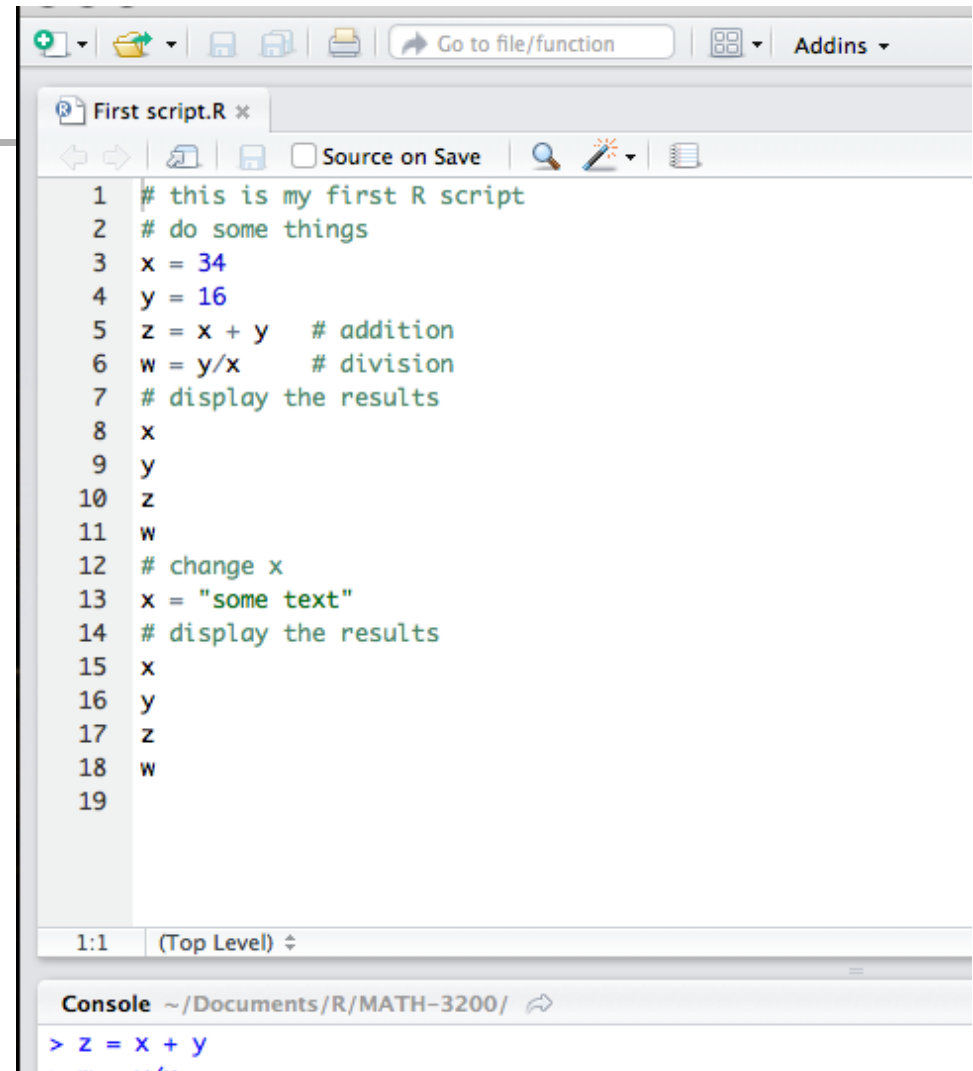
Get started: Creating an R script

- Click File -> **New File -> R Script**
- The script editor opens with an empty script



Get started: Creating an R script

- ```
this is my first R script
do some things
x = 34
y = 16
z = x + y # addition
w = y/x # division
display the results
x
y
z
w
change x
x = "some text"
display the results
x
y
z
w
```



The screenshot shows the RStudio IDE interface. The top toolbar includes icons for creating a new file, saving, and other standard functions. The main editor window, titled 'First script.R', contains the following R code:

```
1 # this is my first R script
2 # do some things
3 x = 34
4 y = 16
5 z = x + y # addition
6 w = y/x # division
7 # display the results
8 x
9 y
10 z
11 w
12 # change x
13 x = "some text"
14 # display the results
15 x
16 y
17 z
18 w
19
```

Below the script editor is the 'Console' window, which shows the output of the first two lines of code:

```
> z = x + y
[1] 50
> w = y/x
[1] 0.4705882
```

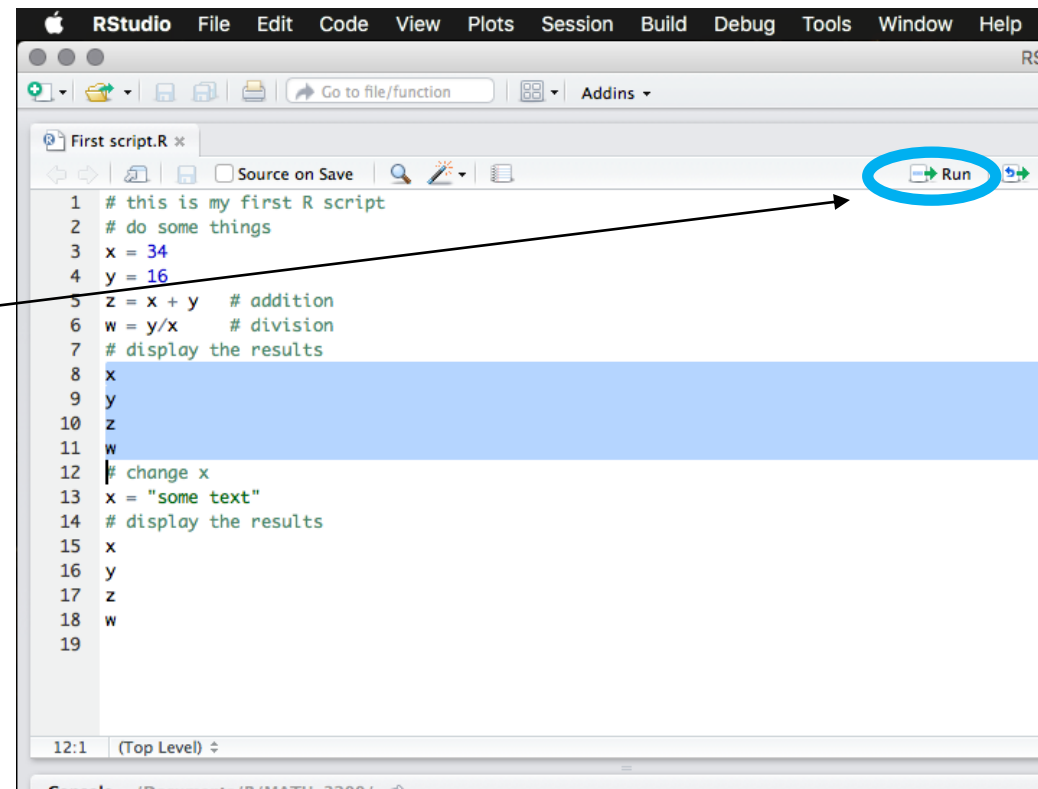
# Get started: R script

- To save: click File -> Save as
- To open: double click on saved script file
- Run code: highlight/place cursor along code line and either:

- click run, or
- Press Ctrl + Enter (windows)

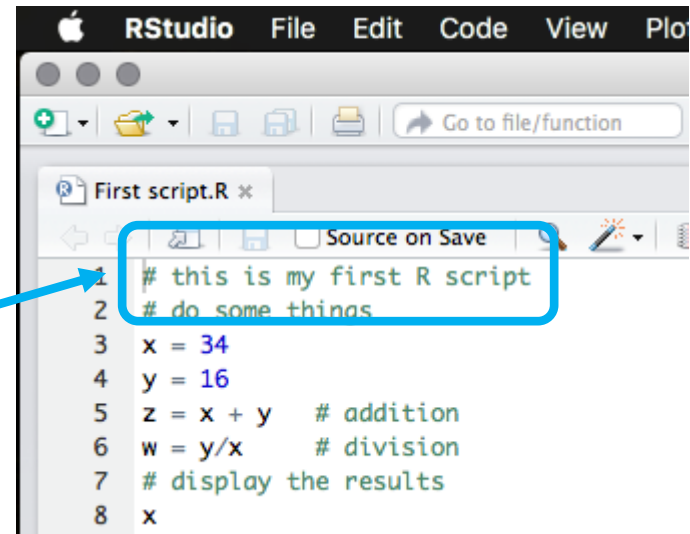


- Press Command + Enter (Mac)

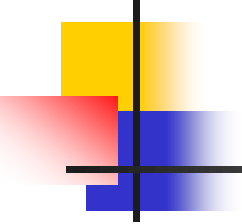


# Get started: R script- comments/documenting your code

- Important to place comments in your code.
- Helps understand and remind what your code is meant to do.
- A comment in R code begins with the **#** symbol.
- **#** is treated as a comment and is ignored during execution.



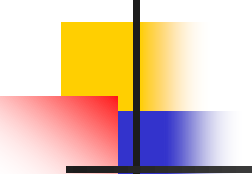
```
1 # this is my first R script
2 # do some things
3 x = 34
4 y = 16
5 z = x + y # addition
6 w = y/x # division
7 # display the results
8 x
```



## Commands, objects and functions

---

- Commands in R are generally made up of two parts: objects and functions. These are
- separated by '`<-`', which you can think of as meaning 'is created from'. As such, the general
- form of a command is:  
`Object<-function`



# Commands, objects and functions

```
R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.
```

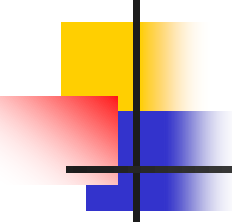
```
Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.
```

```
> Tutors <- c("Lynda", "Patrick", "Collins", "Dase1", "Peter", "James")
```

Object

Function

In the `Console' window the cursor is waiting for you to type in some R commands



# Basic Commands

---

```
x<- c(1, 6, 8, 10, 7, 5, 3, 14, 19, 6, 5, 1, 5, 3)
```

```
max(x, na.rm = TRUE) #maximum
```

```
min(x, na.rm = TRUE) #minimum
```

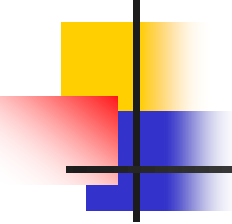
```
mean(x, na.rm = TRUE) #mean
```

```
median(x, na.rm = T) #median
```

```
sum(x, na.rm = T) #sum
```

```
var(x, na.rm = T) #variance
```

```
sd(x, na.rm = T) #standard deviation
```



# Basic commands

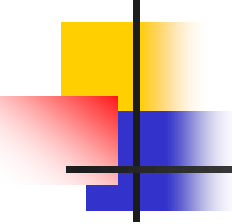
---

`length(x)` #number of items

`summary(x)` #produce result summaries of data models ..

`table(x)` #tabulation or crosstabulation

`x[5]` #List or subsetting- here the 5<sup>th</sup> item of dataset x



# Useful Functions

---

`str(mtcars)`

`class(mtcars)` # class or type of an object

`names(mtcars)` # names

`vector1 <- c(1, 2, 3)` # vector for use in examples below

`vector2 <- c(4, 5, 6)` # vector for use in examples below

`c(vector1, vector2)` # combine objects into a vector

`cbind(vector1, vector2)` # combine objects as columns

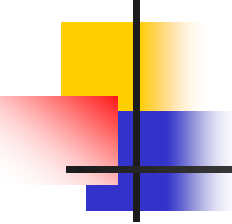
`rbind(vector1, vector2)` # combine objects as rows

`ls()` # list current objects in workspace

`rm(vector1)` # delete an object from workspace

`newobject <- edit(mtcars)` # edit copy and save a newobject

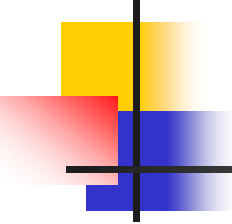
`fix(mtcars)` # edit in place- replace original



# Points to note

---

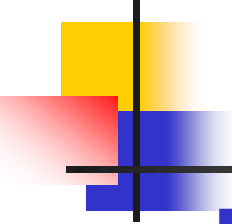
1. R is case sensitive
2. Comment in your codes: #
3. Get help:
  - ?
  - ??
  - help()



# Points to note

---

- Results of calculations can be stored in objects using the assignment operators:
  - An arrow (`<-`) formed by a smaller than character and a hyphen without a space!
  - The equal character (`=`).

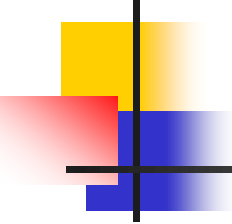


# Points to note

---

- These objects can then be used in other calculations. To print the object just enter the name of the object. There are some restrictions when giving an object a name:
  - Object names cannot contain 'strange' symbols like `!`, `+`, `-`, `#`.
  - A dot (`.`) and an underscore (`_`) are allowed, also a name starting with a dot.
  - Object names can contain a number but cannot start with a number.
  - R is case sensitive, `X` and `x` are two different objects, as well as `temp` and `tempP`.

# Points to note



```
An example
x <- c(1:10)
x[(x>8) | (x<5)]
yields 1 2 3 4 9 10
```

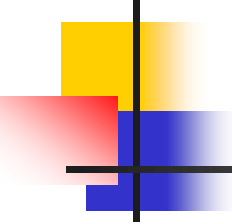
```
How it works
x <- c(1:10)
x
```

```
x > 8
```

```
x < 5
```

```
x > 8 | x < 5
```

```
x[c(T,T,T,T,F,F,F,F,T,T)]
```



# Points to note

---

- To list the objects that you have in your current R session use the function `ls` or the function `objects`.

`ls()`

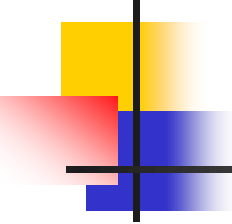
```
[1] "x" "y"
```

- So to run the function `ls` we need to enter the name followed by an opening ( and a closing ). Entering only `ls` will just print the object, you will see the underlying R code of the the function `ls`. Most functions in R accept certain arguments. For example, one of the arguments of the function `ls` is `pattern`. To list all objects starting with the letter `x`:

```
x2 = 9
```

```
y2 = 10
```

```
ls(pattern="x")
```



# Points to note

---

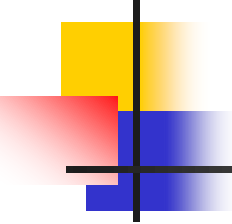
- If you assign a value to an object that already exists then the contents of the object will be overwritten with the new value (without a warning!). Use the function `rm` to remove one or more objects from your session.

```
rm(x, x2)
```

- Lets create two small vectors with data and a scatterplot.

`#concatenate function or c(), groups things together`

```
z2 <- c(1,2,3,4,5,6)
z3 <- c(6,8,3,5,7,1)
plot(z2,z3)
title("My first scatterplot")
```



# Calculations in R

---

```
x = sin(9)/75
```

```
y = log(x) + x^2
```

```
x
```

```
[1] 0.005494913
```

```
y
```

```
[1] -5.203902
```

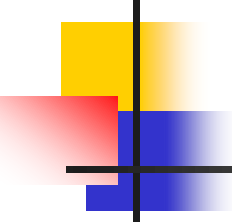
```
m <- matrix(c(1,2,4,1), ncol=2)
```

```
m
```

```
 [,1] [,2]
```

```
[1,] 1 4
```

```
[2,] 2 1
```

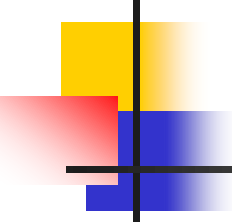


# R Warning !

---

R is a case sensitive language.

TUTORS, Tutors, and  
tutors are three different  
objects



# R Working directory

---

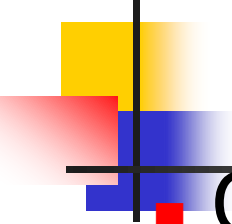
The **working directory** is just a file path on your computer that sets the default location of any files you read into **R**, or save out of **R**.

`getwd()` # print the current working directory

`ls()` # list the objects in the current workspace

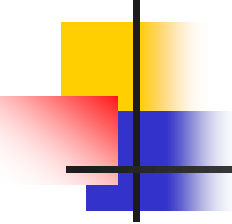
`setwd(mydirectory)` # change to mydirectory

`setwd("c:/docs/mydir")`



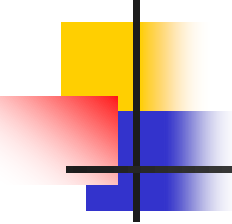
# R Workspace

- Objects created during an R session are held in memory
- A collection of current objects is called the **workspace**.
- This workspace is not saved on disk unless you tell R to do so.
- Objects are lost if R is closed without saving the objects, or worse when R or your system crashes during a session.



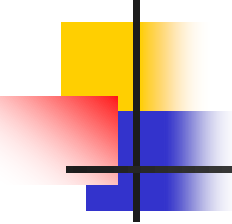
# R Workspace

- When you close the RGui or the R console window, the system will ask if you want to save the workspace image.
- If you select to save the workspace image then all the objects in your current R session are saved in a file **.RData**.
- This is a binary file located in the working directory of R, which is by default the installation directory of R.



# R Workspace

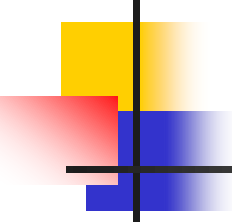
- In Rstudio session you can also explicitly save the workspace image.
- Go to the `*Session*` menu and then select `*Save Workspace As*`, or use the `save.image` function.
  - ## save to the current working directory  
`save.image()`
  - ## just checking what the current working directory is  
`getwd()`
  - ## save to a specific file and location  
`save.image("C:\\Program Files\\R\\R-2.5.0\\bin\\"sessionname.RData")`



# R Workspace

---

- If you have saved a workspace image and you start R the next time you can restore the workspace.
- All your previously saved objects are available again.
- You can also explicitly load a saved workspace, that could be the workspace image of someone else. Go the '*Session*' menu and select '*Load Workspace.*' or "*File*" then "*Open File*"

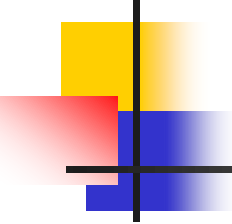


# R console

---

Commands are entered interactively at the **R** user prompt. **Up** and **down arrow keys** scroll through your command history.

You will probably want to keep different projects in different physical directories.



# R computer pathway

---

**R** gets confused if you use a path in your code like

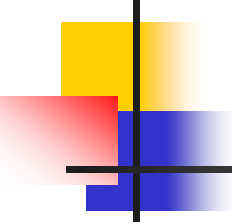
*c:|mydocuments|myfile.txt*

This is because R sees "\" as an escape character. Instead, use

*c:||my documents||myfile.txt*

*or*

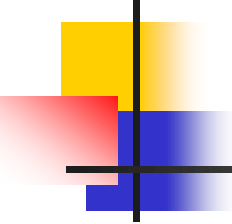
*c:/mydocuments/myfile.txt*



# R Workspace

---

# work with your previous commands  
`history()` # display last 25 commands  
`history(max.show=Inf)` # display all previous commands



# R Workspace

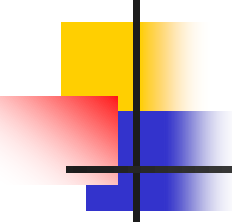
---

# save your command history

`savehistory(file="myfile")` # default is ".Rhistory"

# recall your command history

`loadhistory(file="myfile")` # default is ".Rhistory"



# R Help

---

Once **R** is installed, there is a comprehensive built-in help system. At the program's command prompt you can use any of the following:

`help.start()` # general help

`help(lm)` # help about function *lm*

`?lm` # same thing

`apropos("lm")` # list all function containing string *lm*

`example(lm)` # show an example of function *lm*



# R Help

---

# search for foo in help manuals and archived mailing lists

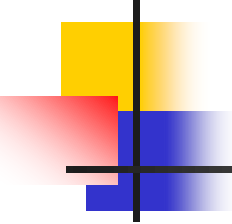
`RSiteSearch("lm")`

# get vignettes on using installed packages

# Vignettes are long-form guide to your package

`vignette()` # show available vignettes

`vignette("lm")` # show specific vignette



# R Packages

---

- A key strength of R is that the system can easily be extended.
- Allows to write new functions and package those functions in a so called 'R package' (or 'R library') – greater than 10000 libraries
- The R package may contain other R objects, for example data sets or documentation.
- E.g. packages for specific topics, drawing maps, exporting objects to html, time series analysis, spatial statistics and the list goes on and on.

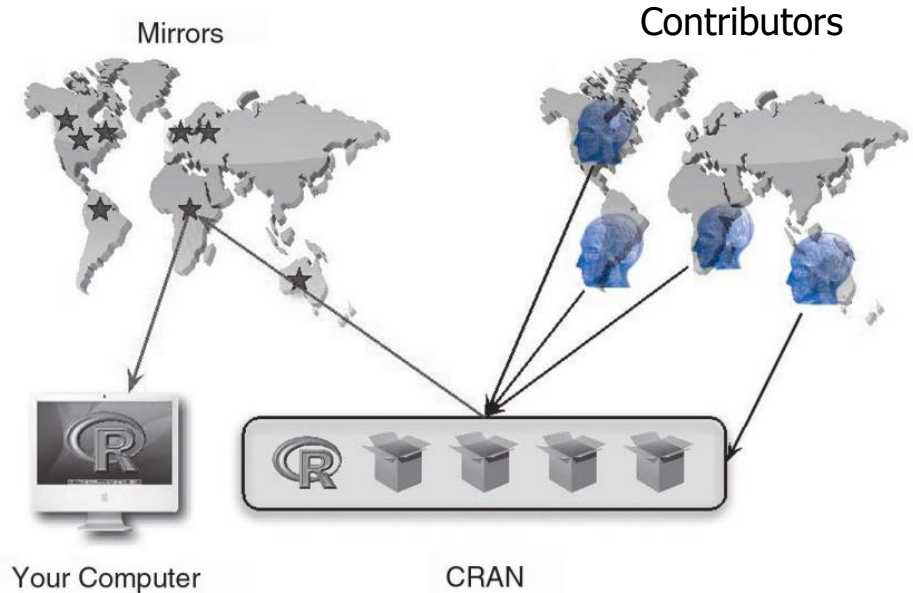
# R-chitecture

At the centre of the diagram is the CRAN: a repository of the base R software and hundreds of packages.

Contributors from all over the world write new packages and upload them into the CRAN for others to use.

The CRAN itself is mirrored at different places across the globe—multiple copies

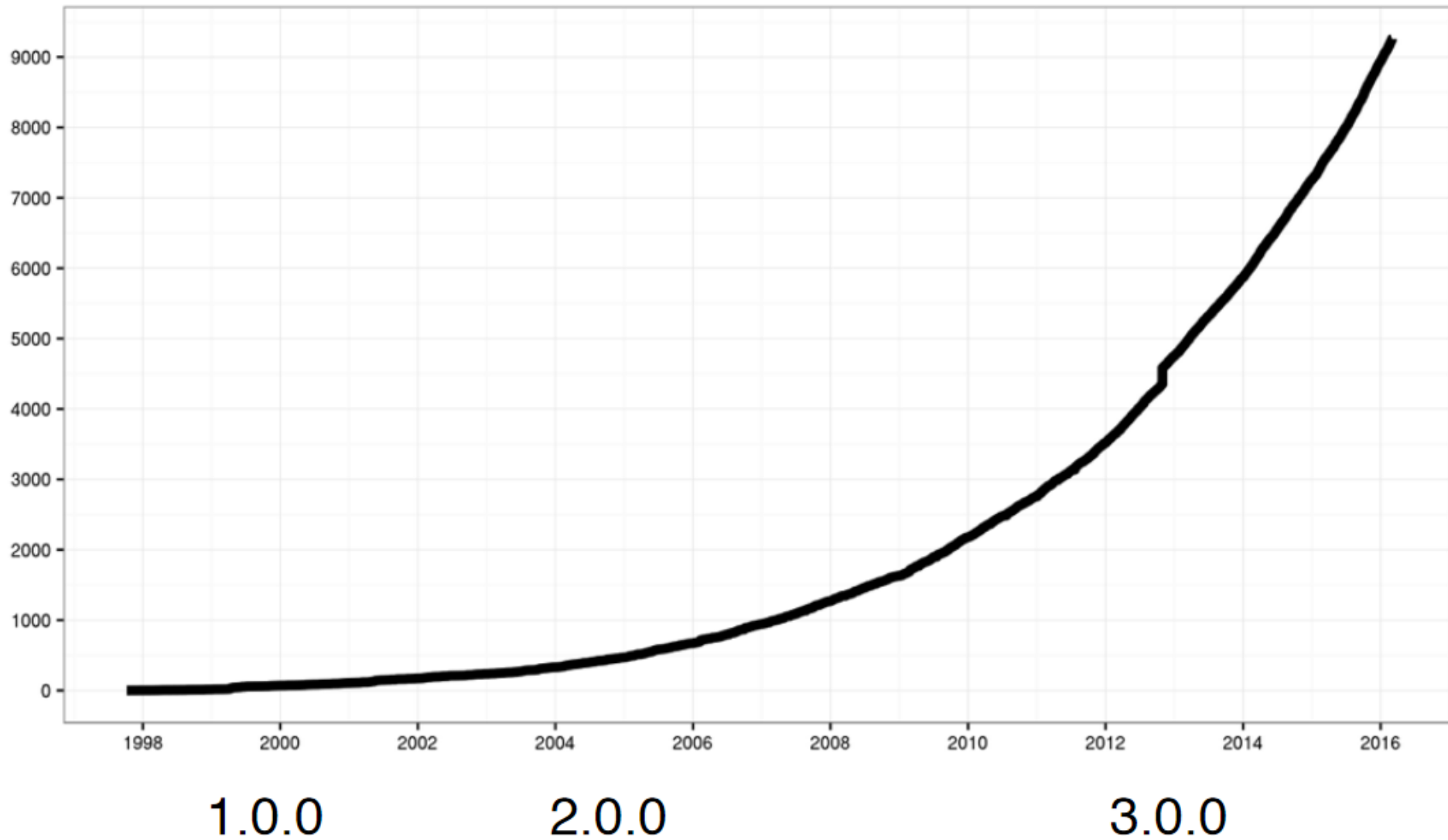
Users download the software, and install packages via CRAN.

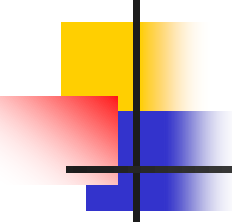


Adapted from Field, Miles, Field, 2012

# R packages

Number of R packages ever published on CRAN



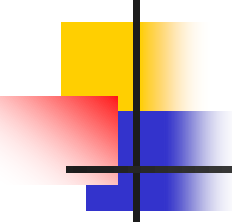


# R Packages

- On installation, already a number (around 30) of packages are downloaded as well.
- To use a function in an R package, that package has to be attached to the system.
- When you start R not all of the downloaded packages are attached, only seven packages are attached to the system by default.
- You can use the function `search()` to see a list of packages that are currently attached to the system, this list is also called the search path.

`search()`

```
[1] ".GlobalEnv" "package:stats" "package:graphics"
[4] "package:grDevices" "package:datasets" "package:utils"
[7] "package:methods" "Autoloads" "package:base"
```



# R Packages

---

To attach another package to the system you can use the menu or the library function. Via the menu:

Select the 'Packages' menu and select 'Load package...', a list of available packages on your system will be displayed. Select one and click 'OK', the package is now attached to your current R session. Via the library function:

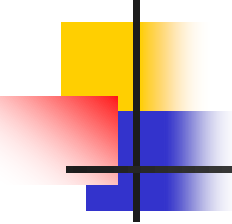
```
library(MASS)
```

```
shoes$A
```

```
[1] 13.2 8.2 10.9 14.3 10.7 6.6 9.5 10.8 8.8 13.3
```

```
shoes$B
```

```
[1] 14.0 8.8 11.2 14.2 11.8 6.4 9.8 11.3 9.3 13.6
```



# R Packages

---

- The function `library()` can also be used to list all the available libraries on your system with a short description. Run the function without any arguments

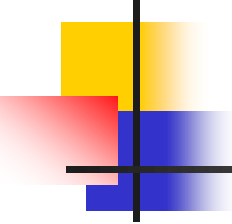
`library()`

Packages in library 'C:/PROGRA~1/R/R-25~1.0/library':

|           |                                                                         |
|-----------|-------------------------------------------------------------------------|
| base      | The R Base Package                                                      |
| Boot      | Bootstrap R (S-Plus) Functions (Canty)                                  |
| class     | Functions for Classification                                            |
| cluster   | Cluster Analysis Extended Rousseeuw et al.                              |
| codetools | Code Analysis Tools for R                                               |
| datasets  | The R Datasets Package                                                  |
| DBI       | R Database Interface                                                    |
| foreign   | Read Data Stored by Minitab, S, SAS,<br>SPSS, Stata, Systat, dBase, ... |
| graphics  | The R Graphics Package                                                  |

R Data Analysis

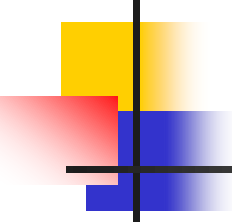
AcademicDataAnalysts.org



# R Packages

---

- Install one package  
`install.packages("graphics")`
- Several packages: just replace the package names  
`install.packages("graphics"); install.packages("moments");`  
`install.packages("graphics")`



# R Datasets

---

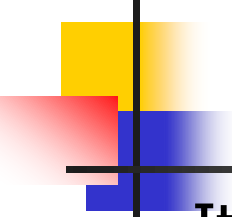
**R** comes with a number of sample datasets that you can experiment with. Type

**data( )**

to see the available datasets. The results will depend on which packages you have loaded. Type

**help(CO2)**

for details on a sample dataset.



# R Conflicting objects

---

- It is not recommended to do, but R allows the user to give an object a name that already exists. If you are not sure if a name already exists, just enter the name in the R console and see if R can find it. R will look for the object in all the libraries (packages) that are currently attached to the R system. R will not warn you when you use an existing name.

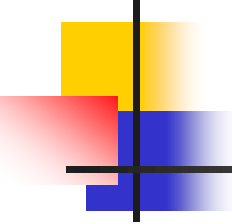
```
mean = 10
```

```
mean
```

```
[1] 10
```

- The object mean already exists in the base package, but is now **masked** by your object mean. To get a **list of all masked objects** use the function conflicts.

```
"body<-" "mean"
```



# R Conflicting objects

---

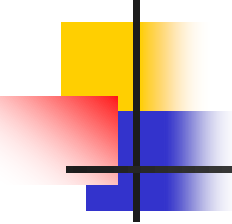
The object mean already exists in the base package, but is now masked by your object mean. To get a list of all masked objects use the function `conflicts()`.

```
conflicts()
```

```
[1] "body<-" "mean"
```

You can safely remove the object mean with the function `rm()` without risking deletion of the mean function.

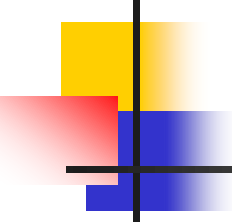
Calling `rm()` removes only objects in your working environment by default.



# Data Types

---

**R** has a wide variety of data types including scalars, vectors (numerical, character, logical), matrices, dataframes, and lists.



# Data types: logical

---

Binary, two possible values  
represented by TRUE and FALSE

```
x = c(3, 7, 1, 2)
```

```
x > 2
```

```
[1] TRUE TRUE FALSE FALSE
```

```
x == 2
```

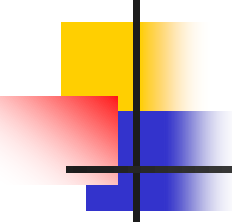
```
[1] FALSE FALSE FALSE TRUE
```

```
!(x < 3)
```

```
[1] TRUE TRUE FALSE FALSE
```

```
which(x > 2)
```

```
[1] 1 2
```



# Data types: character/string

---

Each element in the vector is a string of one or more characters.

Built in character vectors are `letters` and `LETTERS` which provide the 26 lower (and upper) case letters, respectively.

```
y = c("a", "bc", "def")
```

```
length(y)
```

```
[1] 3
```

```
nchar(y)
```

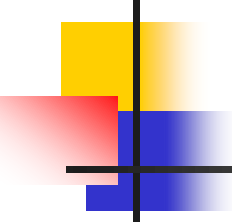
```
[1] 1 2 3
```

```
y == "a"
```

```
[1] TRUE FALSE FALSE
```

```
y == "b"
```

```
[1] FALSE FALSE FALSE
```



# Data types: factor

---

A factor- type vector contains a set of numeric codes with character-valued levels.

Example - a family of two girls (1) and four boys (0),

```
kids = factor(c(1,0,1,0,0,0), levels = c(0, 1),
labels = c("boy", "girl"))
```

```
kids
```

```
[1] girl boy girl boy boy boy
```

```
Levels: boy girl
```

```
class(kids)
```

```
[1] "factor"
```

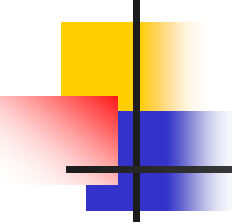
```
mode(kids)
```

```
[1] "numeric"
```

Regardless of the levels/labels of the factor, the numeric storage is an integer with 1 corresponding to the first level (in alphabetical order) etc

```
as.numeric(kids) #b in boy comes before g in girls in the alphabet therefore boy=1
```

```
[1] 2 1 2 1 1 1
```



# Data types: factor

Tell **R** that a variable is **nominal** by making it a factor. The factor stores the nominal values as a vector of integers in the range [ 1... k ] (where k is the number of unique values in the nominal variable), and an internal vector of character strings (the original values) mapped to these integers.

# variable gender with 20 "male" entries and

# 30 "female" entries

```
gender <- c(rep("male",20), rep("female", 30))
```

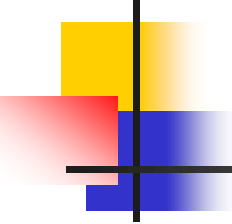
```
gender <- factor(gender)
```

# stores gender as 20 1s and 30 2s and associates

# 1=female, 2=male internally (alphabetically)

# R now treats gender as a nominal variable

```
summary(gender)
```



# Data types: Vectors, Matrices, Arrays, Lists, and Data Frames

---

**Vector** – a collection of **ordered** homogeneous elements.

We can think of matrices, arrays, lists and data frames as deviations from a vector. The deviations are related to the two characteristics **order** and **homogeneity**.

**Matrix** - a vector with two-dimensional shape information.

```
xx = matrix(1:6, nrow=3, ncol =2)
```

```
xx
```

```
 [,1] [,2]
```

```
[1,] 1 4
```

```
[2,] 2 5
```

```
[3,] 3 6
```

```
class(x)
```

```
[1] "numeric"
```

```
is.vector(xx)
```

```
[1] FALSE
```

```
is.matrix(xx)
```

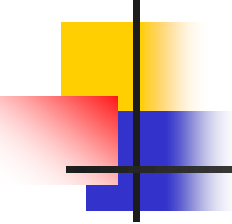
```
[1] TRUE
```

```
length(xx)
```

```
[1] 6
```

```
dim(xx)
```

```
[1] 3 2
```



# Data types: Matrices

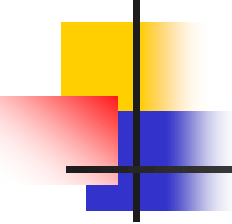
---

All columns in a matrix must have the same mode(numeric, character, etc.) and the same length.

The general format is

```
mymatrix <- matrix(vector, nrow=r, ncol=c,
 byrow=FALSE,dimnames=list(char_vector_rownames
 , char_vector_colnames))
```

**byrow=TRUE** indicates that the matrix should be filled by rows. **byrow=FALSE** indicates that the matrix should be filled by columns (the default). **dimnames** provides optional labels for the columns and rows.



# Data types: Matrices

---

# Example: generates 5 x 4 numeric matrix

```
y<-matrix(1:20, nrow=5,ncol=4)
```

# another example

```
cells <- c(1,26,24,68)
```

```
rnames <- c("R1", "R2")
```

```
cnames <- c("C1", "C2")
```

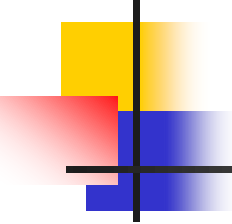
```
mymatrix <- matrix(cells, nrow=2, ncol=2, byrow=TRUE,
dimnames=list(rnames, cnames))
```

#Identify rows, columns or elements using subscripts.

```
mtcars[,4] # 4th column of matrix
```

```
mtcars[3,] # 3rd row of matrix
```

```
mtcars[2:4,1:3] # rows 2,3,4 of columns 1,2,3
```



# Data types: **vectors**

---

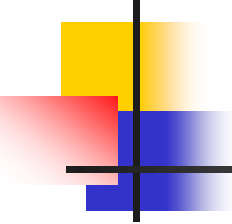
```
a <- c(1,2,5.3,6,-2,4) # numeric vector
```

```
b <- c("one","two","three") # character vector
```

```
c <- c(TRUE,TRUE,TRUE,FALSE,TRUE,FALSE)
#logical vector
```

Refer to elements of a vector using subscripts.

```
a[c(2,4)] # 2nd and 4th elements of vector
```



# Data types: Data frames

A data frame is more general than a matrix, in that different columns can have different modes (numeric, character, factor, etc.).

A list with possible heterogeneous vector elements of the same length. The elements of a data frame can be numeric vectors, factor vectors, and logical vectors, but they must all be of the same length.

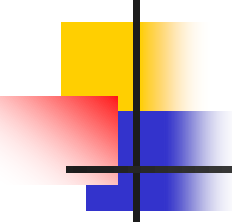
```
d <- c(1,2,3,4)
```

```
e <- c("red", "white", "red", NA)
```

```
f <- c(TRUE,TRUE,TRUE,FALSE)
```

```
mydata <- data.frame(d,e,f)
```

```
names(mydata) <- c("ID","Color","Passed") #variable names
```



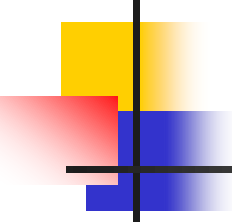
# Data types: Data frames

There are a variety of ways to identify the elements of a dataframe .

`mydata[2:3]` # columns 3,4,5 of dataframe

`mydata[c("Color","Passed")]` # columns Color and Passed from dataframe

`mydata$Color` # variable Passed in the dataframe



# Data types: Lists

---

An ordered collection of objects (components). A list allows you to gather a variety of (possibly unrelated) objects under one name. The elements of a list can be numeric vectors, character vectors, matrices, arrays, and lists.

```
myList = list(a = 1:10, b = "def", c(TRUE, FALSE, TRUE))
```

```
myList$a
```

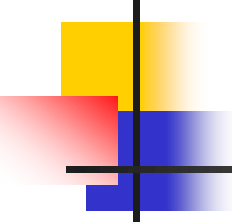
```
[1] 1 2 3 4 5 6 7 8 9 10
```

```
myList$b
```

```
[1] "def"
```

```
myList[[3]]
```

```
[1] TRUE FALSE TRUE
```



# Data types: Lists

---

`length(myList)` # there are 3 elements in the list

`class(myList)` # the class is a "list"

`names(myList)` # are "a", "b" and the empty character ""

`myList[1:2]` # returns a list with two elements

`myList[1]` # returns a **list** with one element. What is  
`length(myList[1])` ?

`myList[[1]]` # returns a **vector** with ten elements, the  
#numbers 1, 2, ..., 10

What is `length(myList[[1]])`